Upstream: what YARG's own rules say, and what we are asking them

The project's aim is that work here be upstreamable rather than a fork that diverges forever. This document records what upstream's process actually is — read from their documentation rather than assumed — and what we have asked them.

Their process, from `CONTRIBUTING.md`

- **Ask on Discord before building a feature.** Their words: "*It's recommended that you ask there before working on a new feature, in case someone is already working on a feature/change.*" Discord: https://discord.gg/sqpu4R552r. Task tracker: https://yarg.youtrack.cloud/agiles/147-7/current.
- **PRs target `dev`. A PR based on `master` will not be accepted.**
- Every feature falls in one of six tiers, and the tier decides whether a PR is even looked at:

Tier	What it means for a PR
In-Development	Do not PR without coordination — they are already on it
Planned	PRs welcome
Stretch Goal	Discouraged, but experimenting and sharing progress is invited, and "*large progress on a stretch goal may promote it to a higher tier*"
Eligible	PRs welcome; they will not build it themselves
Problematic	Heavily discouraged; big headachey impacts elsewhere
Out of Scope	"*Do NOT PR these features. Your PR will immediately be denied*"

Their Out of Scope examples include CON Decryption. That is worth noting: this project independently made CON/mogg decryption a permanent non-goal on legal grounds, and upstream has independently ruled it out on their own. Two of our three permanent non-goals are

things upstream would reject on sight. We are not asking them for anything they have already refused.

Which tier a remote song source falls into is not published, and none of the tier examples resemble it. That is the first question to ask, and it is exactly the question their contributing guide tells contributors to ask.

What we searched before asking

Everything in this section is context for talking to their team, not work this project is taking on. The scope line is in `ROADMAP.md` under "What this project is, and what it is not": we build a library server and a client that talks to YARG, plus the means to manage that library without launching the game. Features that live inside YARG and change how it plays are theirs. They are recorded here because knowing what upstream already has in flight is what makes a conversation with them useful — not because adjacency is a reason to adopt something.

Searched again on 2026-09-07, and the first search was not good enough. It covered issues and missed pull requests entirely, which produced a confident claim that was false. What follows is the corrected picture; the retracted version is kept below it because the error is the interesting part.

No issue or PR proposes loading songs from a remote server. That is the thing this project is actually building, and it is unclaimed. `#1030 "Add support for user-supplied song sources"` is about **source icons**, not sources of songs. `#167 "Online Servers"` is online co-op play. Nothing in `YARG.Core` touches stream loading either — the open PRs there are vocals (#437) and chart parsing (#204), and nothing has moved on `SngFile` since the SNG handling changes of early 2024.

But the queue half of #860 has been attempted, and we said it had not.

- **#860**, open since August 2024, unlabelled: search and queue from a phone *"whilst YARG is running"*.
- **#984, "Add very basic ability to select song via http to music library"**, by `theli-ua`, opened 2025-02-27, **still open as a draft**, one commit, targets `dev`. In the author's words it *"allows to open `yarg_host:9090`, be presented with basic list of songs in the library, upon clicking on them they will become selected in yarg"* — he labels it *"Very basic version of #860"* and *"more of PoC to get some general feedback on the idea"*. It has since collected a `Has Conflicts` label and no reviews.

Read the discussion on it before writing anything about #860. `joewestcott` tried it and replied: *"I've just checked this out and it works really well!"*, that it *"could lay the foundation for a YARG app"* for parties where everyone browses on their own phone, and then asked for a specific architecture — *"it's probably better for YARG itself to serve a JSON API over HTTP, rather than generating HTML within csharp"*, with the frontend bundled as an HTML file served on the same

port: "`yarg_host:9090` *would serve the HTML file from the frontend team, and* `yarg_host:9090/api` *would have a GET endpoint for a song list and a POST endpoint to change the selected song.*" He offered to build that frontend himself. The author's answer was "*I don't know if anyone is actually interested in working on this or of any plans to do anything in this area*", and it has sat there since.

Three things follow, and none of them were visible from the issues alone:

1. **"Nobody has built it" was wrong.** Someone built a working proof of concept eighteen months ago and a contributor confirmed it works. The retracted paragraph is below.
2. **"Nothing outside the game can reach a running client" was the wrong diagnosis.** #984 solves it the obvious way — it puts an HTTP *server* inside the game. That is not a remote song source and it does not need one.
3. **It is direct evidence that networking in the Unity layer is welcome**, which ADR-004 had only inferred from YARG.Core having none. Someone put an HTTP server in the Unity project and the response was enthusiasm plus a request for more endpoints.

Also merged since we last looked: #1540, `XaiaX`, merged into `dev` 2026-07-07 — a "Web Page" option in Settings → File Management → Export Songs that writes the library as a self-contained HTML browser with search, per-instrument filter chips, sortable columns and a detail view (9,366 songs ≈ 1.13 MB). It is a **static export**, so it is neither live nor able to select a song, and it is not the party-mode frontend. It does mean YARG now ships an HTML song browser, and its record schema is prior art for anyone building the one `joewestcott` asked for.

Retracted: what this section said before

> ## 860 has been open since August 2024 … so there is demand and **nobody has built it.** The
>
> reason nobody has is in the request's own words: "*whilst YARG is running*". **Nothing outside the game can reach a running client.** A remote song source is the thing that could.

Both bolded claims are false, and the Discord draft below repeated the first one back to a channel where the two people who did the work are presumably reading. The cause is the usual one for this project: **a negative result from a search that did not cover pull requests, written down as a fact about the world.** The searches that would have caught it took under five minutes.

Two bugs we can hand them, with reproductions

Found 2026-09-08 by pointing our mirror at a deliberately hostile server (a body cut in half mid-transfer, a real archive served under someone else's hash, a 500, and random bytes). Both are in the **same failure path** of `SngFile.TryLoadFromFile` — the branch taken when a file turns out not to be a `.sng` — and both are one-liners. Neither has anything to do with remote libraries, which is what makes them worth sending first.

1. The `FileStream` leaks when the file is not a `.sng`. `IO/SngHandler/SngFile.cs:100`:

```
using var tracker = new SngTracker();
var filestream = new FileStream(filename, FileMode.Open, FileAccess.Read, FileShare.Read,
1);
...
    if (filestream.Read(tag) < tag.Length || !tag.SequenceEqual(SNGPKG))
    {
        return default;          // <-- filestream was never given to the tracker
    }
    tracker.Stream = filestream; // <-- only reached on success
```

The stream is opened before the tag check and only handed to `tracker` after it passes, so the early return drops the only reference to an open handle. `tracker.Dispose()` has nothing to close. On Windows the file stays **locked until the finalizer runs**, so a caller that rejects a file and then tries to delete it gets `The process cannot access the file … because it is being used by another process`. Anything that scans a folder containing one non-`.sng` file and then tries to clean up hits this.

2. `Dispose()` throws on the value a failed load returns. `IsLoaded` is `_tracker != null` and correctly reports `false`, but `Dispose()` is an unconditional `_tracker.Dispose()`. So the natural way to write the caller —

```
using var sng = SngFile.TryLoadFromFile(path, false);
if (!sng.IsLoaded) { throw new Exception("not a readable .sng"); }
```

— throws a `NullReferenceException` from the implicit `Dispose()` as the stack unwinds, and that NRE **replaces** the caller's own exception. We hit this exactly: a clear "downloaded file is not a readable .sng" reached the player as "Object reference not set to an instance of an object".

Both are guarded against on our side (`SongServerSync.VerifyChartHash` no longer uses `using`, and partial downloads are swept on the next run), so neither blocks us. They are offered because they are small, real, and reproducible — and a first contribution that fixes a bug is a better introduction than one that asks for an API.

The patch, and the measurement that says it works

`docs/patches/0001-sngfile-release-file-and-survive-dispose-on-rejected-load.patch` — two lines of code against `028969a9`, `git am`-ready:

```
-            _tracker.Dispose();
+            _tracker?.Dispose();
...
+                    filestream.Dispose();
                     return default;
```

Measured A/B with the hostile-server probe, one variable changed and nothing else:

YARG.Core	`.part` files left after a rejected download
stock `028969a9`	**1** — the file is locked, the delete fails
patched	**0** — deleted immediately

The rejected download's reported reason is the real one in both cases only because our side stopped using `using`; with stock YARG.Core and the obvious caller shape it reads "Object reference not set to an instance of an object".

The patch is **not applied to our submodule** — it sits on a local branch and the pointer is back on `028969a9`, so the fork builds against stock upstream and our workarounds stay in place. It exists to be sent, not to be depended on.

The shape of the ask, and why it is smaller than it sounds

The server already hands out **plain `.sng` files that unmodified YARG reads natively** — that is the design commitment in ADR-001, and it has been demonstrated on two machines, two operating systems and two CPU architectures. So we are not asking upstream to support a protocol in order to play our songs. They already can.

What a player cannot do **in stock YARG** is get those songs without running a separate sync tool. So the upstream ask is narrow: a way for YARG to discover and fetch songs from a URL.

Our fork already does it, which changes what this post is asking for. It is not "would you build this" but "we built it, in the Unity layer, touching nothing in YARG.Core — here is the shape, would you take it, and which of the two seams below would you want first". A working implementation is on the table rather than a proposal.

The ask got smaller once the code was read

The paragraph that used to sit here said the seam was that `SongEntry` is abstract while `ActualLocation`, `SortBasedLocation` and `GetLastWriteTime()` assume a local path. True, and not useful — it names a symptom, and anyone who maintains YARG.Core would know that already. `ADR-004` replaced it with what the code actually says, and the ask that falls out is much smaller and much more concrete:

- **`SngFile` has exactly one loader**, `TryLoadFromFile(string, bool)` (`IO/SngHandler/SngFile.cs:100`), and it opens a `FileStream` itself. There is no stream-taking overload — but the method is **already stream-oriented after its first handful of lines**: it wraps the file in either `YARGSongFileStream` or the raw `FileStream`, assigns `tracker.Stream`, and everything after that reads only from that stream. So `TryLoadFromStream` is an extract-method, not a redesign. `FixedArray` already reads from streams (`ReadRemainder(Stream)`, `Read(Stream, long, bool)`).
- **`SngEntry`, `UnpackedIniEntry` and their base `IniSubEntry` are all `internal`** with private constructors, so nobody outside the assembly can add an entry type. Anything at the entry level has to happen inside YARG.Core, by them or with them.
- **One source seam already exists and does not go far enough**: `protected abstract FixedArray<byte>? GetChartData(string filename)` (`SongEntry.IniBase.cs:82`). `LoadChart()` is genuinely source-agnostic on top of it; everything else is not — `SngEntry` reaches for `SngFile.TryLoadFromFile(_location, …)` at **seven** sites and the local filesystem at nine more.
- **YARG.Core contains no networking at all** — zero matches for `HttpClient|System.Net|UnityWebRequest` across the library. So the right place for a fetch is the Unity layer, which already networks, and **we are not asking them to put an `HttpClient` in YARG.Core.** Saying that out loud is worth a sentence, because it is the objection a maintainer would reach for first.

So the post below asks for **two small things** rather than for a feature: `TryLoadFromStream`, which is useful on its own and has no remote-library baggage, and — separately, lower confidence — whether a materialise-before-load hook on `IniSubEntry` is the kind of thing they would ever want.

Everything above was measured against `yarg` `3673672` / `YARG.Core` `028969a` on 2026-09-07, and should be re-checked before the post goes out if that is much later. Citing a line number that has moved is a bad first impression in a channel full of people who know the file.

Draft: the Discord post

Not yet sent. Jay sends it, under his own account; nothing goes out without him.

Hey folks — I've been building a self-hosted song server for YARG and wanted to ask here before going further, per CONTRIBUTING.

The short version: a small Go server that indexes a song folder and hands out plain `.sng` files. It is not a game modification — **unmodified YARG already reads everything it serves**, and I've had that running across two machines, two OSes and two CPU architectures. Today a little sync client pulls songs into a folder and YARG scans it like any other folder. The obvious next step is YARG pointing at a server URL directly, instead of needing a separate tool.

I read through `YARG.Core` and the Unity side before writing this, so I can ask something more specific than "would you like this feature". Five questions, smallest first:

1. Would you take `SngFile.TryLoadFromStream(Stream, bool)` on its own? `TryLoadFromFile` opens the `FileStream` itself, but everything after the first few lines already works purely off `tracker.Stream` — so this looks like an extract-method rather than a redesign, and `FixedArray` already has stream readers. It's useful for anything that has `.sng` bytes without a path on disk, remote or not. If that's welcome I'd happily open that PR by itself and leave everything below for later.

2. Is a "materialise before load" hook on `IniSubEntry` something you'd ever want? Lower confidence on this one. `GetChartData` is already an abstract seam and `LoadChart` is source-agnostic on top of it, but the other loaders go through `SngFile.TryLoadFromFile(_location, …)` — seven call sites in `SngEntry` — so a one-line hook the load methods call first, defaulting to a no-op, seems less invasive than abstracting `_location`. Very open to being told that's the wrong shape.

3. Would a second "delivered out of band" song folder be objectionable in principle? `SongContainer.RunRefresh` already appends `PathHelper.SetlistPath` to the scan list — a folder the player didn't add, populated by the Launcher, skipped cleanly when absent. What I'd be adding is a second producer for that same shape of folder. If that pattern is fine, most of this needs nothing from `YARG.Core` at all.

4. What should a release build do about plain HTTP? `insecureHttpOption` is `NotAllowed`, and a song server on someone's LAN is plain HTTP — so the feature simply doesn't work in a release build as things stand. I've set it to `DevelopmentOnly` in my fork to get the thing testable, which ships nothing weaker to players, but that's postponing the question rather than answering it. Relax it, require HTTPS from the server, or make it an opt-in per host? I'd rather match whatever you'd want than pick one and find out later.

5. Which tier does a remote song source fall into? It doesn't match any of the CONTRIBUTING examples. As far as I can find nothing proposes fetching songs from a server — the closest things are #860 and #984, which are search/queue and song *selection* from a phone rather than a source of songs. Happy to stay out of the way if someone is on it and I've missed them.

> To be explicit about what I'm *not* asking for: **no `HttpClient` in `YARG.Core`.** There's no networking in the library at all right now and I don't think this needs to change that — the fetching belongs in the Unity layer, which already networks. Also no CON decryption, no touching `songcache.bin`, and no distributing copyrighted audio. The server serves what the operator already has.
>
> I'm building it in my fork either way, so this isn't a request for anyone to do work. I'd just rather build it in a shape you'd consider than find out later it was never going to fit.
>
> One last thing, and it's a question rather than a pitch. I found **#984** after I'd drafted most of this — the PoC that serves a song list on `yarg_host:9090` and selects a song when you tap it — and the discussion on it (a JSON API on `/api`, an HTML frontend bundled alongside, GET for the list and POST to change the selection) describes almost exactly the API my server already speaks, just pointed at the local library instead of a remote one. It's been sitting in draft with conflicts for a while and the author wasn't sure anyone was interested.
>
> To be clear about scope: **that side of it isn't what I'm building.** Mine is a library server — one copy of the library on the network, every machine plays from it, and you can browse and manage it from a phone without launching the game at all. Controlling a running client is a different feature and it's yours.
>
> I mention it only because the API #984's discussion describes — GET a song list, POST to change the selection — is close to what my server already speaks, just pointed at a remote library instead of a local one. **It's LGPL-3.0 and it's all public**, so if it's useful to whoever picks that up, take any of it. And if the two turn out to be one conversation rather than two, I'd rather know that now than after I've built the second half of something in the wrong place.
>
> Everything's LGPL-3.0-or-later, same as YARG: https://github.com/Coffeehedake/yarg-song-server

One rule about that link

The post links the public GitHub mirror, never `gitlab.badassium.com`. Origin is a GitLab instance on a home server; GitHub is a downstream, force-overwritten mirror that exists precisely so there is something public to point at. Pasting the origin URL into a Discord with a few thousand strangers in it would advertise home infrastructure for no benefit — the mirror carries identical content.

Verified before this draft went anywhere: `Coffeehedake/yarg-song-server` is public, LGPL-3.0, and its last push matches the last push to origin, so the mirror is live rather than a stale snapshot from months ago.

What to do with the answer

Question 1 is the one that matters most, and it is deliberately separable. A yes to `TryLoadFromStream` is a merged PR and a working relationship with upstream regardless of what happens to the rest; a no to everything else costs nothing we were counting on, because `ADR-004` increment 1 needs no YARG.Core change at all.

- **In-Development or someone is on it** — stop, and offer what we have to whoever is.
- **Planned or Eligible** — design toward a PR from the start, and target `dev`.
- **Stretch Goal** — build it in the fork and share progress; their own rule says large progress can promote a tier.
- **Problematic or Out of Scope** — build it in the fork for ourselves, drop the upstreaming goal for this feature specifically, and say so plainly in the ROADMAP rather than leaving a dead aspiration in it.

Their answer does not block Phase 3. It shapes it.
